

Topology-Informed Visual Prompting for Hierarchical Vision-Language-Action Control Supplementary Material

Haoyang Wu Abhinav Kumar Dmitry Berenson
Robotics Department, University of Michigan, Ann Arbor, MI, USA
{haoyangw, abhin, dmitryb}@umich.edu

A Related Works

Generalist VLA Policies Generalist robot policies learn behaviors across tasks and embodiments [1, 2, 3], while VLA models connect pretrained vision-language representations to robot actions [4, 5, 6]. π_0 uses a flow-matching action expert for continuous control [6], and $\pi_{0.5}$ adds heterogeneous co-training across robot, semantic, and web data [7]. GR00T N1 targets generalist humanoid control [8], MolmoAct introduces explicit spatial trajectory reasoning [9], and MolmoBot uses large-scale simulated demonstrations for zero-shot transfer [10]. Despite these advances, policies trained primarily through nominal action supervision need not distinguish global topological relationships when locally similar observations require different motions. Our experiments show that directly fine-tuned π_0 and $\pi_{0.5}$ policies can be improved by an explicit topology-reasoning layer.

Agentic Robot Architectures Agentic robot systems separate task-level reasoning from physical execution through an intermediate representation that a continuous controller can execute. Code as Policies exposes perception and control APIs through executable programs [11], while VoxPoser grounds language-model reasoning in compositional 3D value maps for motion generation [12]. Hierarchical VLA systems provide learned interfaces to continuous control: RT-H predicts language motions before robot actions [13], HAMSTER communicates image-space trajectories to a low-level policy [14], and Hi Robot uses a VLM to interpret instructions and feedback before invoking VLA skills [15]. These systems make the representation communicated across the agent-policy interface a central design choice. Image-space annotations instantiate such an interface by communicating demonstrated trajectories, motion traces, candidate actions, and compact control cues to vision-language models and robot policies [16, 17, 14, 18, 19]. They provide interpretable guidance without modifying the downstream policy architecture. We study topology as the reasoning abstraction behind this interface: a VLM estimates the topological state and transmits correction waypoints as visual prompts, a VLA executes the corresponding continuous actions, and the hierarchy updates after each action chunk. We compare against PEEK [18], which masks regions judged irrelevant and draws an end-effector path, but does not explicitly preserve or reason about task-relevant topology.

Topology-Guided Planning and Data Generation Topological abstractions such as homotopy and homology classes have guided motion planning in constrained environments [20, 21, 22]. Loop-based representations and the Gauss linking integral have also characterized grasping, claspings, hooking, cooperative manipulation, and deformable-object manipulation [23, 24, 25, 26, 27, 28]. These methods generally require obstacle geometry during execution. We instead use geometry in approximate simulation to identify topological mismatches, define recovery objectives, and generate targeted correction data. At deployment, the agent infers topology from images. Topology therefore serves both as an online reasoning abstraction for guiding the VLA and as an offline abstraction for directing simulation toward recovery data that nominal demonstrations omit.

B Virtual h-Signature Construction

The topological structures of interest are homotopy classes of task-relevant loops, defined in Mitrano et al. [22]. Two loops belong to the same homotopy class if one can be continuously deformed into the other without intersecting an obstacle. For manipulation, we construct loops from task-relevant robot, object, and obstacle geometry. We compute this signature by extending the h-signature, defined as the Gauss linking number between a task-relevant robot-object loop and a task-relevant obstacle loop. The linking number is an integer that measures the signed winding of the two closed loops through each other [20]. It is invariant to continuous deformation and can change only when loops pass through one another.

The physical robot and object paths in our tasks are open, so we extend the h-signature by connecting endpoints with virtual edges. As shown in Fig. 1 of the main paper, we first trace paths along the robot and manipulated object. Each robot path terminates at an end effector, which is paired with the endpoint of the object path at the corresponding grasp or contact site. These paths and endpoint pairings remain fixed across configurations of the task. At each configuration x_t , a straight virtual edge connects each paired endpoint. For example, for a bimanual grasp through two handles, the two handles are the endpoints of the object path, and the virtual edges connect them to the corresponding end effectors. The physical paths and virtual edges together form the closed robot-object loop L_t . We represent the task-relevant environmental structure by a separate closed obstacle loop O . The virtual edges represent candidate straight-line end-effector paths to their target contact sites. Including these paths in L_t lets the signature indicate whether straight-line motion is compatible with the demonstrated virtual h-signature or whether the robot must first route around an obstacle.

For a configuration x_t , its virtual h-signature $h^v = \text{Lk}(L_t, O)$, where Lk is the Gauss Linking integral as used in [20]. We define two configurations x and x' as having the same topological signature if and only if $h^v(x) = h^v(x')$. The same definition applies across tasks; only the physical paths, endpoint pairings, and obstacle loop used to construct L_t and O change with the task. This signature can therefore represent, for example, whether a cable passes through an aperture or whether a robot-object path is routed around an obstacle, independently of their poses and local geometry. A virtual edge might pass through an obstacle, for example, if an end-effector is below the rack shown in Fig. 1 of the main paper. Abstracting the scene into the L_t and O loops still allows us to compute h^v and use it to guide the policy, for example moving the end-effector above the rack to achieve the demonstrated h-signature.

C Evaluation Tasks

We consider three tasks in simulation: Rope Pulling, Cable Threading, and Box Pickup. For all tasks, we design simulation environments and task execution oracles to autonomously generate data for VLA and VLM fine-tuning.

C.1 Rope Pulling

Fig. 1(a) shows a representative initial observation. We evaluate each method over 50 trials with randomized rope, goal, and robot configurations. Because the rope tip begins beyond the robot’s reachable workspace, the robot alternates arms to grasp and pull it into the goal region. Since $h_t^v = 0$ throughout, no signature correction is required. This task tests whether our pipeline remains compatible with general learning without degrading nominal performance.

C.2 Cable Threading

Fig. 1(b) shows a representative initial observation. We evaluate each method over 100 simulation trials. Cable Threading requires a transition from $h_0^v = 0$, where the cable has not passed through the target loop, to $h^{v*} = 1$. Its free tip is initially held by the right gripper on one side of the loop, with the left gripper near the opposite side. We randomize the cable configuration, robot configuration,

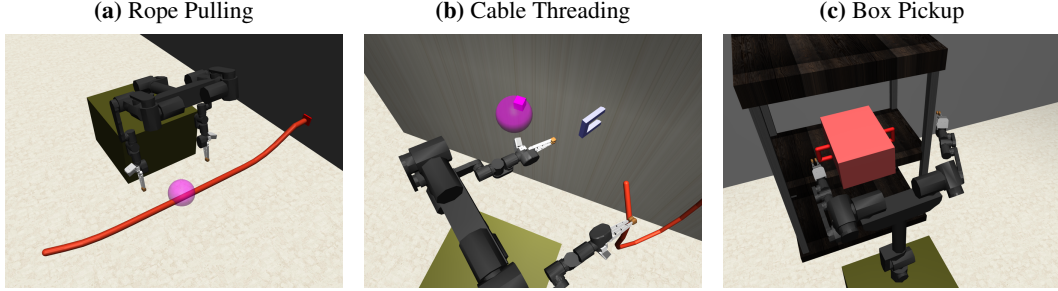


Figure 1: Representative initial simulation observations for (a) Rope Pulling, (b) Cable Threading, and (c) Box Pickup.

and target-loop position across trials. Because moving the tip around the loop may bring it near the goal without achieving the required signature, the oracle passes the tip through the loop with the right gripper, secures it with the left gripper, and releases it with the right gripper. The left arm then pulls the tip toward the goal. A trial succeeds when the cable remains through the loop and its tip reaches the purple goal region without being dropped.

C.3 Box Pickup

Fig. 1(c) shows an initial observation in simulation. We evaluate each method over 100 simulation trials and 30 real-world trials. Box Pickup requires correcting $h_0^v = 1$ to $h^{v*} = 0$ before lifting the box. The box begins within a rack, with both grippers outside its two handle openings. We randomize the rack size, box position, and robot configuration across trials. In trials with $h_0^v = 1$, the right arm lies between the rack posts, so moving directly toward its handle would cause a collision. The robot must first route the arm around the posts to reach $h_t^v = 0$, then approach the handles, pass each gripper through its corresponding opening, and lift the box. A trial succeeds when the box clears the rack while maintaining the demonstrated signature and both grasps.

D Implementation Details

We use π_0 [6] and $\pi_{0.5}$ [7] as VLA backbones and Qwen3-VL-8B-Instruct [29] for both VLM calls. For each backbone, the base policy, PEEK, and our method use the same VLA training configuration.

We fine-tune each VLA for 20,000 steps with a global batch size of 32, a learning rate of 5×10^{-5} , and 1,000 warm-up steps. We use a 10-step action horizon, 224×224 images, bfloat16 precision, and LoRA [30] ranks of 16 for the 2B vision-language backbone and 32 for the 300M action expert. Training uses four NVIDIA A40 GPUs. During execution, all 10 actions in a predicted chunk are completed before the next VLM and VLA update.

We fine-tune the VLM for h^v and waypoint prediction for one epoch using LoRA [30] with rank 8 and $\alpha = 16$. Training uses a global batch size of 32 across four NVIDIA A40 GPUs, a 4,096-token context, at most 262,144 image pixels, FP16 precision, and a cosine schedule with 10% warm-up. The learning rate is 5×10^{-5} for Box Pickup and 1×10^{-4} for Cable Threading.

E Ablation Results

We ablate the two roles of topology using the $\pi_{0.5}$ backbone. The No Visual Prompting variant retains the full topology-aware training dataset but omits VLM-generated prompts at deployment, isolating topology’s role as an execution-time interface between the VLM and VLA. Removing this interface reduces success by 12, 8, and 7 percentage points on Rope Pulling, Cable Threading, and Box Pickup, respectively. The Task-Continuation Data Only variant instead disables topology-aware data generation: signature mismatches do not trigger targeted corrections, and topology does not specify recovery objectives. The nominal dataset is augmented only with task-continuation

Table 1: Ablation task-success rates (%) in simulation using the $\pi_{0.5}$ backbone.

Method	Rope Pulling	Cable Threading	Box Pickup
Full Method (Ours)	64	77	82
No Visual Prompting	52	69	75
Task-Continuation Data Only	N/A	24	54

trajectories. Thus, this ablation not only contains fewer trajectories, but also prevents topology from determining which recovery data should be generated. Success decreases from 77% to 24% on Cable Threading and from 82% to 54% on Box Pickup, showing that topology is a useful abstraction for generating data that addresses failures requiring a signature transition. This ablation is not applicable to Rope Pulling because that task requires no signature correction.

F Real-World Adaptation Protocol

To run the Box Pickup task on hardware, we apply visual domain randomization to the colors and textures of scene objects and to the camera positions and viewing angles. To address residual sim-to-real mismatch in reaching the box handles, we collect 15 short real-world trajectories demonstrating precise approaches to the handles for lifting the box. We also collect three h^v -correction trajectories to reduce the risk of overfitting to the reaching demonstrations. Both the baselines and our VLA are further fine-tuned on these 18 trajectories, whose total frame count is 3% of that in the collected simulation data.

For VLM adaptation, we human-label the h^v signature in observations from 1,000 random real-world configurations. We fine-tune the VLM on these labeled observations, which constitute approximately 5% of its total training data. During real-world execution, once the end effectors reach the designated handle regions, a low-level controller directly lifts the box. We do not fine-tune the PEEK VLM, as doing so would require full demonstrations on hardware which we do not assume we have. As such, our method does not rely on hardware tele-operation data which may be challenging to collect, especially for bimanual manipulation. At the same time, we can utilize cheaper-to-collect h^v -annotated data.

G Qualitative Prompt and Recovery Results

G.1 Simulation

Our method overlays topology-informed motion prompts on the original observation, preserving both the manipulation target and the surrounding obstacle geometries that constrain how it can be reached. The relevance of those geometries differs across tasks. Rope Pulling requires repeated grasping and pulling without modifying the topological signature. In Cable Threading and Box Pickup, however, the relationship between the robot-object loop and the obstacle loop determines whether the robot can approach the target directly or must first modify h^v .

The comparison with PEEK [18] illustrates the consequences of excluding this scene context. PEEK [18] emphasizes manipulated objects and subtrajectory goals but can mask geometry required for signature reasoning. Its masks omit the obstacle loop in Cable Threading and the relevant rack posts in Box Pickup, as shown in Fig. 2(a). Our method preserves this geometry in Fig. 2(b). Rope Pulling has no task-relevant obstacle loop and is consequently less sensitive to this loss of context, helping explain PEEK’s smaller performance drop. Together, these results suggest that visual guidance should preserve structures that constrain the motion, even when they are not themselves manipulation targets.

G.2 Real World

Fig. 2 compares the visual prompts produced by PEEK and our method. In configurations with an incorrect signature, the nominal VLA frequently reaches directly toward the handle without first moving around the rack post. This behavior resembles the direct approach demonstrated from training configurations with the demonstrated signature, but leads to obstruction in these real world initial states. The VLM trained using PEEK [18] similarly directs the end effector toward the handle without providing guidance around the post. In contrast, our method recognizes the incorrect signature and guides the arm around the post before approaching the handle.

We also observe recovery from incorrect signature predictions during execution. In some trials, our method initially predicts the signature incorrectly and directs the end effector toward the handle. At a subsequent observation update, it revises the prediction and redirects the arm around the post. This feedback-driven revision provides evidence that repeated signature estimation and prompt updates can correct an initially inappropriate motion.

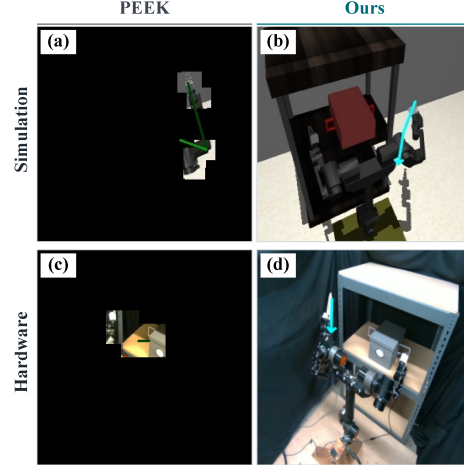


Figure 2: Visual prompting for Box Pickup in simulation with (a) PEEK and (b) ours, and on hardware with (c) PEEK and (d) ours. PEEK masks out task-relevant rack geometry, whereas our method preserves this context and provides appropriate visual prompts

References

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [2] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *ICRA*, 2024.
- [3] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [7] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke,

- A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054>.
- [8] NVIDIA, :, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
 - [9] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. VanderBilt, A. Farhadi, D. Fox, and R. Krishna. Molmoact: Action reasoning models that can reason in space, 2025. URL <https://arxiv.org/abs/2508.07917>.
 - [10] A. Deshpande, M. Guru, R. Hendrix, S. Jauhri, A. Eftekhar, R. Tripathi, M. Argus, J. Salvador, H. Fang, M. Wallingford, W. Pumacay, Y. Kim, Q. Pfeifer, Y.-C. Lee, P. Wolters, O. Rayyan, M. Zhang, J. Duan, K. Farley, W. Han, E. Vanderbilt, D. Fox, A. Farhadi, G. Chalvatzaki, D. Shah, and R. Krishna. Molmob0t: Large-scale simulation enables zero-shot manipulation, 2026. URL <https://arxiv.org/abs/2603.16861>.
 - [11] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation*, 2023.
 - [12] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. In *Conference on Robot Learning*, 2023.
 - [13] S. Belkhale, T. Ding, T. Xiao, P. Sermanet, Q. Vuong, J. Tompson, Y. Chebotar, D. Dwibedi, and D. Sadigh. RT-H: Action hierarchies using language. In *Robotics: Science and Systems*, 2024.
 - [14] Y. Li, Y. Deng, J. Zhang, J. Jang, M. Memmel, C. Garrett, F. Ramos, D. Fox, A. Li, A. Gupta, et al. Hamster: Hierarchical action models for open-world robot manipulation. In *ICLR*, 2025.
 - [15] L. X. Shi, B. Ichter, M. R. Equi, L. Ke, K. Pertsch, Q. Vuong, J. Tanner, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In *International Conference on Machine Learning*, 2025.
 - [16] J. Gu, S. Kirmani, P. Wohlhart, Y. Lu, M. G. Arenas, K. Rao, W. Yu, C. Fu, K. Gopalakrishnan, Z. Xu, et al. Rt-trajectory: Robotic task generalization via hindsight trajectory sketches. *arXiv preprint arXiv:2311.01977*, 2023.
 - [17] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *ICLR*, 2025.
 - [18] J. Zhang, M. Memmel, K. Kim, D. Fox, J. Thomason, F. Ramos, E. Bıyık, A. Gupta, and A. Li. Peek: Guiding and minimal image representations for zero-shot generalization of robot manipulation policies. *arXiv preprint arXiv:2509.18282*, 2025.
 - [19] S. Nasiriany, F. Xia, W. Yu, T. Xiao, J. Liang, I. Dasgupta, A. Xie, D. Driess, A. Wahid, Z. Xu, et al. Pivot: Iterative visual prompting elicits actionable knowledge for vlms. *arXiv preprint arXiv:2402.07872*, 2024.
 - [20] S. Bhattacharya, M. Likhachev, and V. Kumar. Topological constraints in search-based robot path planning. *Autonomous Robots*, 2012.

- [21] F. T. Pokorny, D. Kragic, L. E. Kavraki, and K. Goldberg. High-dimensional winding-augmented motion planning with 2d topological task projections and persistent homology. In *ICRA*, 2016.
- [22] P. Mitrano and D. Berenson. The grasp loop signature: A topological representation for manipulation planning with ropes and cables. In *ICRA*, 2024.
- [23] F. T. Pokorny, J. A. Stork, and D. Kragic. Grasping objects with holes: A topological approach. In *ICRA*, 2013.
- [24] J. A. Stork, F. T. Pokorny, and D. Kragic. A topology-based object representation for clasping, latching and hooking. In *Humanoids*, 2013.
- [25] A. Marzinotto, J. A. Stork, D. V. Dimarogonas, and D. Kragic. Cooperative grasping through topological object representation. In *Humanoids*, 2014.
- [26] J. A. Stork, F. T. Pokorny, and D. Kragic. Integrated motion and clasp planning with virtual linking. In *IROS*, 2013.
- [27] W. H. Lui and A. Saxena. Tangled: Learning to untangle ropes with rgb-d perception. In *IROS*, 2013.
- [28] M. Sudry, T. Jurgenson, A. Tamar, and E. Karpas. Hierarchical planning for rope manipulation using knot theory and a learned inverse model. In *CoRL*, 2023.
- [29] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [30] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.